



university of
 groningen

Models and Semantics of Computation (MaSC)

Lecture 6: Abstract Semantics and Lambda Notation

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands



- ▶ Big-step semantics for IMP
- ▶ General proof principle: Well-founded induction
- ▶ Instances: Structural and rule induction
- ▶ Application: Proving determinacy for IMP

Today:

- ▶ Program equivalence: concrete and parametric equivalence
- ▶ Lambda notation (and the λ -calculus)

Program Equivalence



- ▶ **Abstract semantics** refers to the notion of behavior that we use to compare programs for equivalence
 - ▶ We can compare programs for equivalence or inequality. We can consider
 - ▶ fully specified programs on unspecified (initial) memories
- OR
- ▶ programs with symbolic (=generic) arithmetic and boolean expressions, and symbolic commands
 - ▶ In either case, we carry out the derivations using the operational semantics

Program Equivalence



- ▶ Symbolic manipulation of programs is possible if we can guarantee that their semantics is preserved
- ▶ Useful to study program transformations (interpreters, compilers, code optimization, refactoring, etc)
- ▶ But, when are two commands equivalent?

Operational Equivalence



A first attempt, using the **default initial memory** σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Operational Equivalence



A first attempt, using the **default initial memory** σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

- Note: $\sim_o$ is defined using a universal quantification (on σ).
Hence, to show that $c_1 \not\sim_o c_2$ it suffices to find one counterexample (for σ).
- We consider equivalence under a **context**: an environment for commands.

Operational Equivalence



A first attempt, using the **default initial memory** σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

- Note: $\sim_o$ is defined using a universal quantification (on σ).
Hence, to show that $c_1 \not\sim_o c_2$ it suffices to find one counterexample (for σ).
- We consider equivalence under a **context**: an environment for commands.
- A context has a **hole**, denoted $\bullet$: we can obtain new commands by “plugging” arbitrary commands in $\bullet$. Examples of contexts:

$$C_1[\bullet] \triangleq [\bullet] \quad \rightsquigarrow C_1[c] = c, \quad \text{for any } c \text{ (empty context)}$$

$$C_2[\bullet] \triangleq y := 1; [\bullet] \quad \rightsquigarrow C_2[c] = y := 1; c, \quad \text{for any } c$$

$$C_3[\bullet] \triangleq \text{while } b \text{ do } [\bullet]; x := 0 \quad \rightsquigarrow C_3[c] = \text{while } b \text{ do } c; x := 0, \quad \text{for any } c$$

Operational Equivalence



A first attempt, using the **default initial memory** σ_0 :

$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

- Note: $\sim_o$ is defined using a universal quantification (on σ).
Hence, to show that $c_1 \not\sim_o c_2$ it suffices to find one counterexample (for σ).
- We consider equivalence under a **context**: an environment for commands.
- A context has a **hole**, denoted $\bullet$: we can obtain new commands by “plugging” arbitrary commands in $\bullet$. Examples of contexts:

$$C_1[\bullet] \triangleq [\bullet] \quad \rightsquigarrow C_1[c] = c, \quad \text{for any } c \text{ (empty context)}$$

$$C_2[\bullet] \triangleq y := 1; [\bullet] \quad \rightsquigarrow C_2[c] = y := 1; c, \quad \text{for any } c$$

$$C_3[\bullet] \triangleq \text{while } b \text{ do } [\bullet]; x := 0 \quad \rightsquigarrow C_3[c] = \text{while } b \text{ do } c; x := 0, \quad \text{for any } c$$

- A **congruence** is an equivalence that is preserved by every context.

Operational Equivalence



$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Example

Consider the commands $x := y + 1$ and $x := 1$.

Operational Equivalence



$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Example

Consider the commands $x := y + 1$ and $x := 1$.

- Under the empty context $C_1[\bullet] \triangleq [\bullet]$, we clearly have: $x := y + 1 \sim_o x := 1$

Operational Equivalence



$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Example

Consider the commands $x := y + 1$ and $x := 1$.

- ▶ Under the empty context $C_1[\bullet] \triangleq [\bullet]$, we clearly have: $x := y + 1 \sim_o x := 1$
- ▶ Now consider $C_2[\bullet] \triangleq y := 1; [\bullet]$. We obtain:

$$C_2[x := y + 1] \triangleq y := 1; x := y + 1 \qquad C_2[x := 1] \triangleq y := 1; x := 1$$

Operational Equivalence



$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Example

Consider the commands $x := y + 1$ and $x := 1$.

- ▶ Under the empty context $C_1[\bullet] \triangleq [\bullet]$, we clearly have: $x := y + 1 \sim_o x := 1$
- ▶ Now consider $C_2[\bullet] \triangleq y := 1; [\bullet]$. We obtain:

$$C_2[x := y + 1] \triangleq y := 1; x := y + 1 \qquad C_2[x := 1] \triangleq y := 1; x := 1$$

- ▶ However, the resulting commands are **not equivalent**, as we have

$$y := 1; x := y + 1 \not\sim_o y := 1; x := 1$$

which we can check by exploiting the operational semantics.

Operational Equivalence



$$c_1 \sim_o c_2 \triangleq \forall \sigma. (\langle c_1, \sigma_0 \rangle \longrightarrow \sigma \Leftrightarrow \langle c_2, \sigma_0 \rangle \longrightarrow \sigma)$$

Example

Consider the commands $x := y + 1$ and $x := 1$.

- ▶ Under the empty context $C_1[\bullet] \triangleq [\bullet]$, we clearly have: $x := y + 1 \sim_o x := 1$
- ▶ Now consider $C_2[\bullet] \triangleq y := 1; [\bullet]$. We obtain:

$$C_2[x := y + 1] \triangleq y := 1; x := y + 1 \qquad C_2[x := 1] \triangleq y := 1; x := 1$$

- ▶ However, the resulting commands are **not equivalent**, as we have

$$y := 1; x := y + 1 \not\sim_o y := 1; x := 1$$

which we can check by exploiting the operational semantics.

- ▶ There is a context that **does not preserve** the equivalence!

Operational Equivalence



We should be more general and use **any initial memory**:

$$c_1 \sim c_2 \triangleq \forall \sigma, \sigma'. (\langle c_1, \sigma \rangle \longrightarrow \sigma' \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma')$$

Operational Equivalence



We should be more general and use **any initial memory**:

$$b_1 \sim b_2 \triangleq \forall \sigma, v. (\langle b_1, \sigma \rangle \longrightarrow v \Leftrightarrow \langle b_2, \sigma \rangle \longrightarrow v)$$

$$c_1 \sim c_2 \triangleq \forall \sigma, \sigma'. (\langle c_1, \sigma \rangle \longrightarrow \sigma' \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma')$$

Operational Equivalence



We should be more general and use **any initial memory**:

$$a_1 \sim a_2 \triangleq \forall \sigma, n. (\langle a_1, \sigma \rangle \longrightarrow n \Leftrightarrow \langle a_2, \sigma \rangle \longrightarrow n)$$

$$b_1 \sim b_2 \triangleq \forall \sigma, v. (\langle b_1, \sigma \rangle \longrightarrow v \Leftrightarrow \langle b_2, \sigma \rangle \longrightarrow v)$$

$$c_1 \sim c_2 \triangleq \forall \sigma, \sigma'. (\langle c_1, \sigma \rangle \longrightarrow \sigma' \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma')$$

Operational Equivalence



We should be more general and use **any initial memory**:

$$a_1 \sim a_2 \triangleq \forall \sigma, n. (\langle a_1, \sigma \rangle \longrightarrow n \Leftrightarrow \langle a_2, \sigma \rangle \longrightarrow n)$$

$$b_1 \sim b_2 \triangleq \forall \sigma, v. (\langle b_1, \sigma \rangle \longrightarrow v \Leftrightarrow \langle b_2, \sigma \rangle \longrightarrow v)$$

$$c_1 \sim c_2 \triangleq \forall \sigma, \sigma'. (\langle c_1, \sigma \rangle \longrightarrow \sigma' \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma')$$

This definition of $\sim$ is

- ▶ an **equivalence** (reflexive, symmetric, transitive)
- ▶ a **congruence**: it is preserved by every context!

Different Kinds of Proofs



1. **Simple (in)equivalence**: fully specified (concrete) programs on unspecified memories. We analyze cases derived from the different **path conditions**.

Different Kinds of Proofs



1. **Simple (in)equivalence**: fully specified (concrete) programs on unspecified memories. We analyze cases derived from the different **path conditions**.
2. **Parametric equivalence**: as before, but considering programs with symbolic (unspecified) expressions a and b and commands c , which act as parameters.

Different Kinds of Proofs



1. **Simple (in)equivalence**: fully specified (concrete) programs on unspecified memories. We analyze cases derived from the different **path conditions**.
2. **Parametric equivalence**: as before, but considering programs with symbolic (unspecified) expressions a and b and commands c , which act as parameters.
3. **Inequality proofs**: programs that can exhibit the same or different behaviors depending on the initial memory.

Different Kinds of Proofs



1. **Simple (in)equivalence**: fully specified (concrete) programs on unspecified memories. We analyze cases derived from the different **path conditions**.
2. **Parametric equivalence**: as before, but considering programs with symbolic (unspecified) expressions a and b and commands c , which act as parameters.
3. **Inequality proofs**: programs that can exhibit the same or different behaviors depending on the initial memory.
4. **Diverging computations**: we consider a rephrased definition of equivalence

$$\forall \sigma, \sigma'. \begin{cases} \langle c_1, \sigma \rangle \longrightarrow \sigma' & \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma' \\ \langle c_1, \sigma \rangle \not\longrightarrow & \Leftrightarrow \langle c_2, \sigma \rangle \not\longrightarrow \end{cases}$$

Concrete Equivalence



$w \triangleq \text{while } x > 0 \text{ do } x := x - 1$
 $c \triangleq \text{if } x > 0 \text{ then } x := 0 \text{ else skip}$
 $c \stackrel{?}{\sim} w$

Concrete Equivalence


$$\begin{aligned} w &\triangleq \text{while } x > 0 \text{ do } x := x - 1 & c &\stackrel{?}{\sim} w \\ c &\triangleq \text{if } x > 0 \text{ then } x := 0 \text{ else skip} \end{aligned}$$

Take an arbitrary σ , and assume $\sigma(x) \leq 0$ (a path condition).

Concrete Equivalence



$$\begin{aligned} w &\triangleq \text{while } x > 0 \text{ do } x := x - 1 & c &\stackrel{?}{\sim} w \\ c &\triangleq \text{if } x > 0 \text{ then } x := 0 \text{ else skip} \end{aligned}$$

Take an arbitrary σ , and assume $\sigma(x) \leq 0$ (a path condition).

$$\begin{aligned} \langle c, \sigma \rangle &\longrightarrow \sigma' \swarrow \langle x > 0, \sigma \rangle \longrightarrow \text{ff}, \langle \text{skip}, \sigma \rangle \longrightarrow \sigma' \\ &\swarrow * \langle \text{skip}, \sigma \rangle \longrightarrow \sigma' \\ &\swarrow_{\sigma' = \sigma} \square \end{aligned}$$

$$\boxed{\langle c, \sigma \rangle \longrightarrow \sigma}$$

Concrete Equivalence



$$\begin{aligned} w &\triangleq \text{while } x > 0 \text{ do } x := x - 1 & c &\stackrel{?}{\sim} w \\ c &\triangleq \text{if } x > 0 \text{ then } x := 0 \text{ else skip} \end{aligned}$$

Take an arbitrary σ , and assume $\sigma(x) \leq 0$ (a path condition).

$$\begin{aligned} \langle c, \sigma \rangle &\longrightarrow \sigma' \swarrow \langle x > 0, \sigma \rangle \longrightarrow \text{ff}, \langle \text{skip}, \sigma \rangle \longrightarrow \sigma' \\ &\swarrow * \langle \text{skip}, \sigma \rangle \longrightarrow \sigma' \\ &\swarrow_{\sigma'=\sigma} \square \end{aligned}$$

$$\boxed{\langle c, \sigma \rangle \longrightarrow \sigma}$$

$$\begin{aligned} \langle w, \sigma \rangle &\longrightarrow \sigma' \swarrow_{\sigma'=\sigma} \langle x > 0, \sigma \rangle \longrightarrow \text{ff} \\ &\swarrow * \square \end{aligned}$$

$$\boxed{\langle w, \sigma \rangle \longrightarrow \sigma}$$

Concrete Equivalence (contd.)



$$\begin{aligned} w &\triangleq \text{while } x > 0 \text{ do } x := x - 1 & c &\stackrel{?}{\sim} w \\ c &\triangleq \text{if } x > 0 \text{ then } x := 0 \text{ else skip} \end{aligned}$$

Take an arbitrary σ , and now assume $\sigma(x) > 0$ (the other path condition).

$$\begin{aligned} \langle c, \sigma \rangle &\longrightarrow \sigma' \nwarrow \langle x > 0, \sigma \rangle \longrightarrow \text{tt}, \langle x := 0, \sigma \rangle \longrightarrow \sigma' \\ &\nwarrow^* \langle x := 0, \sigma \rangle \longrightarrow \sigma' \\ &\nwarrow^*_{\sigma' = \sigma[0/x]} \square \end{aligned}$$

$$\boxed{\langle c, \sigma \rangle \longrightarrow \sigma[0/x]}$$

$$\forall n > 0. P(n) \quad P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$$

A Dedicated Proof



$$\forall n > 0. P(n) \quad P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$$

► **Base case:** $P(1) \triangleq \forall \sigma. \sigma(x) = 1 \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$

Take an arbitrary σ and assume $\sigma(x) = 1$.

$$\langle w, \sigma \rangle \longrightarrow \sigma' \searrow \langle x > 0, \sigma \rangle \longrightarrow \text{tt},$$

$$\langle x := x - 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\searrow^* \langle x := x - 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma'$$

$$\searrow^*_{\sigma'' = \sigma[0/x]} \langle w, \sigma[0/x] \rangle \longrightarrow \sigma'$$

$$\searrow^*_{\sigma' = \sigma[0/x]} \langle x > 0, \sigma \rangle \longrightarrow \text{ff}$$

$$\searrow^* \square$$

$$\boxed{\langle w, \sigma \rangle \longrightarrow \sigma[0/x]}$$

A Dedicated Proof (contd.)



$$\forall n > 0. P(n) \quad P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$$

- ▶ **Inductive step:** $\forall n > 0. P(n) \Rightarrow P(n + 1)$
- ▶ Assume $P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$.

A Dedicated Proof (contd.)



$$\forall n > 0. P(n) \quad P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$$

- ▶ **Inductive step:** $\forall n > 0. P(n) \Rightarrow P(n + 1)$
- ▶ Assume $P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$.
- ▶ Assuming an arbitrary σ with $\sigma(x) = n + 1$, we prove $P(n + 1) \triangleq \forall \sigma. \sigma(x) = n + 1 \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$.
$$\begin{aligned} \langle w, \sigma \rangle &\longrightarrow \sigma' \searrow \langle x > 0, \sigma \rangle \longrightarrow \text{tt}, \langle x := x - 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma' \\ &\searrow^* \langle x := x - 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma' \\ &\searrow^*_{\sigma'' = \sigma[n/x]} \langle w, \sigma[n/x] \rangle \longrightarrow \sigma' \end{aligned}$$

A Dedicated Proof (contd.)



$$\forall n > 0. P(n) \quad P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$$

- ▶ **Inductive step:** $\forall n > 0. P(n) \Rightarrow P(n + 1)$
- ▶ Assume $P(n) \triangleq \forall \sigma. \sigma(x) = n \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$.
- ▶ Assuming an arbitrary σ with $\sigma(x) = n + 1$, we prove $P(n + 1) \triangleq \forall \sigma. \sigma(x) = n + 1 \Rightarrow \langle w, \sigma \rangle \longrightarrow \sigma[0/x]$.

$$\begin{aligned} \langle w, \sigma \rangle &\longrightarrow \sigma' \searrow \langle x > 0, \sigma \rangle \longrightarrow \text{tt}, \langle x := x - 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma' \\ &\searrow^* \langle x := x - 1, \sigma \rangle \longrightarrow \sigma'', \langle w, \sigma'' \rangle \longrightarrow \sigma' \\ &\searrow_{\sigma'' = \sigma[n/x]}^* \langle w, \sigma[n/x] \rangle \longrightarrow \sigma' \end{aligned}$$
- ▶ By the IH, we know that $\langle w, \sigma[n/x] \rangle \longrightarrow \sigma[0/x]$.
By determinacy it must be that $\sigma' = \sigma[0/x]$. Hence, $\langle w, \sigma \rangle \longrightarrow \sigma[0/x]$.

Parametric Equivalence



$$\begin{array}{l} w \triangleq \text{while } b \text{ do } c \\ ww \triangleq \text{while } b \text{ do } \underbrace{\text{while } b \text{ do } c}_w \end{array} \quad w \stackrel{?}{\sim} ww$$

Take an arbitrary σ . First assume $\langle b, \sigma \rangle \longrightarrow \text{ff}$.

$$\langle w, \sigma \rangle \longrightarrow \sigma' \nwarrow_{\sigma'=\sigma} \langle b, \sigma \rangle \longrightarrow \text{ff}$$

$\nwarrow \square$

$$\boxed{\langle w, \sigma \rangle \longrightarrow \sigma}$$

$$\langle ww, \sigma \rangle \longrightarrow \sigma' \nwarrow_{\sigma'=\sigma} \langle b, \sigma \rangle \longrightarrow \text{ff}$$

$\nwarrow \square$

$$\boxed{\langle ww, \sigma \rangle \longrightarrow \sigma}$$

Parametric Equivalence



$$\text{while } b \text{ do } c \stackrel{?}{\sim} \text{while } b \text{ do while } b \text{ do } c$$

Take an arbitrary σ . Now assume both $\langle b, \sigma \rangle \rightarrow \text{tt}$ and $\langle w, \sigma \rangle \rightarrow \sigma'$.

$$\langle ww, \sigma \rangle \rightarrow \sigma'_1 \swarrow \langle b, \sigma \rangle \rightarrow \text{tt},$$

$$\langle w, \sigma \rangle \rightarrow \sigma'', \langle ww, \sigma'' \rangle \rightarrow \sigma'_1$$

$$\swarrow^* \langle w, \sigma \rangle \rightarrow \sigma'', \langle ww, \sigma'' \rangle \rightarrow \sigma'_1$$

$$\swarrow_{\sigma''=\sigma'}^* \langle ww, \sigma' \rangle \rightarrow \sigma'_1$$

$$\swarrow_{\sigma'_1=\sigma'}^* \langle b, \sigma' \rangle \rightarrow \text{ff}$$

$$\swarrow^* \square$$

$$\boxed{\langle ww, \sigma \rangle \rightarrow \sigma'}$$

$$\forall b, c, \sigma, \sigma'. \langle \text{while } b \text{ do } c, \sigma \rangle \rightarrow \sigma' \Rightarrow \langle b, \sigma \rangle \rightarrow \text{ff}$$

Parametric Equivalence



$$\text{while } b \text{ do } c \stackrel{?}{\sim} \text{while } b \text{ do while } b \text{ do } c$$

Take an arbitrary σ . Assume both $\langle b, \sigma \rangle \longrightarrow \text{tt}$ and $\langle w, \sigma \rangle \not\longrightarrow$.

► We have:

$$\begin{aligned} \langle ww, \sigma \rangle &\longrightarrow \sigma'_1 \nwarrow \langle b, \sigma \rangle \longrightarrow \text{tt}, \\ &\quad \langle w, \sigma \rangle \longrightarrow \sigma'', \langle ww, \sigma'' \rangle \longrightarrow \sigma'_1 \\ &\nwarrow^* \langle w, \sigma \rangle \longrightarrow \sigma'', \langle ww, \sigma'' \rangle \longrightarrow \sigma'_1 \end{aligned}$$

► At this point, the program will **diverge** because by hypothesis $\langle w, \sigma \rangle \not\longrightarrow$.
Hence, $\langle ww, \sigma \rangle \not\longrightarrow$.

A Note on Divergence



- Consider the commands:

$w_1 \triangleq \text{while tt do skip}$

$w_2 \triangleq x := 0; \text{while } x \geq 0 \text{ do } x := x + 1$

$w_3 \triangleq y := 0; \text{while } y \leq 0 \text{ do } y := y - 100$

$w_4 \triangleq y := x + 1; \text{while } x \neq y \text{ do } (x := x - 1; y := y + 1)$

- We may ask:

$$w_1 \stackrel{?}{\sim} w_2 \qquad w_1 \stackrel{?}{\sim} w_3 \qquad w_1 \stackrel{?}{\sim} w_4$$

A Note on Divergence



- Consider the commands:

$w_1 \triangleq \text{while tt do skip}$

$w_2 \triangleq x := 0; \text{while } x \geq 0 \text{ do } x := x + 1$

$w_3 \triangleq y := 0; \text{while } y \leq 0 \text{ do } y := y - 100$

$w_4 \triangleq y := x + 1; \text{while } x \neq y \text{ do } (x := x - 1; y := y + 1)$

- We may ask:

$$w_1 \stackrel{?}{\sim} w_2 \qquad w_1 \stackrel{?}{\sim} w_3 \qquad w_1 \stackrel{?}{\sim} w_4$$

A Note on Divergence



- Consider the commands:

$$w_1 \triangleq \text{while } \texttt{tt} \text{ do skip}$$

$$w_2 \triangleq x := 0; \text{while } x \geq 0 \text{ do } x := x + 1$$

$$w_3 \triangleq y := 0; \text{while } y \leq 0 \text{ do } y := y - 100$$

$$w_4 \triangleq y := x + 1; \text{while } x \neq y \text{ do } (x := x - 1; y := y + 1)$$

- We may ask:

$$w_1 \stackrel{?}{\sim} w_2 \quad w_1 \stackrel{?}{\sim} w_3 \quad w_1 \stackrel{?}{\sim} w_4$$

- Let us state $\sim$ as follows

$$\forall \sigma, \sigma'. \begin{cases} \langle c_1, \sigma \rangle \longrightarrow \sigma' & \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma' \\ \langle c_1, \sigma \rangle \not\longrightarrow & \Leftrightarrow \langle c_2, \sigma \rangle \not\longrightarrow \end{cases}$$

A Note on Divergence



- Consider the commands:

$$w_1 \triangleq \text{while } \texttt{tt} \text{ do skip}$$

$$w_2 \triangleq x := 0; \text{while } x \geq 0 \text{ do } x := x + 1$$

$$w_3 \triangleq y := 0; \text{while } y \leq 0 \text{ do } y := y - 100$$

$$w_4 \triangleq y := x + 1; \text{while } x \neq y \text{ do } (x := x - 1; y := y + 1)$$

- We may ask:

$$w_1 \stackrel{?}{\sim} w_2$$

✓

$$w_1 \stackrel{?}{\sim} w_3$$

✓

$$w_1 \stackrel{?}{\sim} w_4$$

✓

- Let us state $\sim$ as follows

$$\forall \sigma, \sigma'. \begin{cases} \langle c_1, \sigma \rangle \longrightarrow \sigma' & \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma' \\ \langle c_1, \sigma \rangle \not\longrightarrow & \Leftrightarrow \langle c_2, \sigma \rangle \not\longrightarrow \end{cases}$$

A Note on Divergence



- Consider the commands:

$$w_1 \triangleq \text{while tt do skip}$$

$$w_2 \triangleq x := 0; \text{while } x \geq 0 \text{ do } x := x + 1$$

$$w_3 \triangleq y := 0; \text{while } y \leq 0 \text{ do } y := y - 100$$

$$w_4 \triangleq y := x + 1; \text{while } x \neq y \text{ do } (x := x - 1; y := y + 1)$$

- We may ask:

$$w_1 \stackrel{?}{\sim} w_2$$

✓

$$w_1 \stackrel{?}{\sim} w_3$$

✓

$$w_1 \stackrel{?}{\sim} w_4$$

✓

- Let us state $\sim$ as follows

$$\forall \sigma, \sigma'. \begin{cases} \langle c_1, \sigma \rangle \longrightarrow \sigma' & \Leftrightarrow \langle c_2, \sigma \rangle \longrightarrow \sigma' \\ \langle c_1, \sigma \rangle \not\longrightarrow & \Leftrightarrow \langle c_2, \sigma \rangle \not\longrightarrow \end{cases}$$

- Any two (always) divergent commands are equivalent!



Lambda Notation

Lambda Notation



Associated to the λ -calculus, the paradigmatic model of sequential computation, centered around functions.

Lambda-notation is useful for at least two reasons:

- ▶ For giving a denotational semantics to IMP
- ▶ For defining higher-order functional languages

Lambda Notation: Key Ingredients



Anonymous functions

$$\lambda x. e$$

where x serves as a formal parameter in e

- ▶ a function that waits for a value to be substituted for x and then evaluates e

Applications

$$e_1 e_2$$

where e_2 is the argument passed to the function e_1

- ▶ denotes the application of the function e_1 to e_2
- ▶ we avoid the parenthesis $e_1(e_2)$

Defining Functions



Given $f(x) \triangleq x^2 - 2 \cdot x + 5$ we can then define

$$f \triangleq \lambda x. (x^2 - 2 \cdot x + 5)$$

(unnecessary parentheses, added for clarity)

Associative Rules and Scoping



- ▶ application is **left-associative**:

$e_1 e_2 e_3$ is read $(e_1 e_2) e_3$

- ▶ abstraction is **right-associative**:

$\lambda x. \lambda y. \lambda z. e$ is read $\lambda x. (\lambda y. (\lambda z. e))$

Also, the **scope** of x in $\lambda x. e$ is e .

That is, x is not visible outside e : it is a 'local variable'.

Alpha Conversion



The names of formal parameters are inessential.

Example

The following two expressions denote the same function:

$$\lambda x.(x^2 - 2 \cdot x + 5) \quad \text{and} \quad \lambda y.(y^2 - 2 \cdot y + 5)$$

Alpha Conversion



The names of formal parameters are inessential.

Example

The following two expressions denote the same function:

$$\lambda x. (x^2 - 2 \cdot x + 5) \quad \text{and} \quad \lambda y. (y^2 - 2 \cdot y + 5)$$

Under suitable conditions, we will have

$$\lambda x. e \equiv \lambda y. e[y/x]$$

where $[y/x]$ denotes **capture-avoiding** substitution (to be formalized later).

Application (Beta Rule)



The application of a function: evaluation via substitution

$$(\lambda x. e) e_0 \equiv e[e_0/x]$$

Example

$\lambda x. \lambda y. (x^2 - 2 \cdot y + 5)$ a function

$(\lambda x. \lambda y. (x^2 - 2 \cdot y + 5)) 2$ its application

$\equiv$

$\lambda y. (2^2 - 2 \cdot y + 5)$ its evaluation — still a function!

Higher-Order Functions



Example

$\lambda f. \lambda x. (x^2 + f\ 1)$ a function

$(\lambda f. \lambda x. (x^2 + f\ 1)) (\lambda y. (2 \cdot y))$ its application (the argument is a function!)

$\equiv$

$\lambda x. (x^2 + (\lambda y. (2 \cdot y))\ 1)$ its evaluation

Another Example



Example

$\lambda f. \lambda x. (x^2 + f\ 1)$ a function

$(\lambda f. \lambda x. (x^2 + f\ 1)) (\lambda y. (2 \cdot y))\ 3$ its application

$\equiv$

$\lambda x. (x^2 + (\lambda y. (2 \cdot y))\ 1)\ 3$ its evaluation

$\equiv$

$3^2 + (\lambda y. (2 \cdot y))\ 1$

$\equiv$

$3^2 + 2 \cdot 1 = 11$

A Simple λ -calculus



- Syntax of expressions:

$$e ::= x \mid \lambda x. e \mid (e_0 e_1)$$

- Notions of α -conversion and β -rule are as before.
- The set of **free variables** of an expression:

$$\text{fv}(x) \triangleq \{x\}$$

$$\text{fv}(\lambda x. e) \triangleq \text{fv}(e) \setminus \{x\}$$

$$\text{fv}(e_0 e_1) \triangleq \text{fv}(e_0) \cup \text{fv}(e_1)$$

An Attempt at Substitution



$$y[e/x] \triangleq \begin{cases} e & \text{if } y = x \\ y & \text{if } y \neq x \end{cases}$$

$$(\lambda y. e')[e/x] \triangleq \begin{cases} \lambda y. e' & \text{if } y = x \\ \lambda y. (e'[e/x]) & \text{if } y \neq x \end{cases}$$

$$(e_0 \ e_1)[e/x] \triangleq (e_0[e/x]) (e_1[e/x])$$

An Attempt at Substitution



$$y[e/x] \triangleq \begin{cases} e & \text{if } y = x \\ y & \text{if } y \neq x \end{cases}$$
$$(\lambda y. e')[e/x] \triangleq \begin{cases} \lambda y. e' & \text{if } y = x \\ \lambda y. (e'[e/x]) & \text{if } y \neq x \end{cases}$$
$$(e_0 e_1)[e/x] \triangleq (e_0[e/x]) (e_1[e/x])$$

Consider $e \triangleq \lambda x. \lambda y. (x^2 + 2 \cdot y + 5)$

An Attempt at Substitution



$$y[e/x] \triangleq \begin{cases} e & \text{if } y = x \\ y & \text{if } y \neq x \end{cases} \quad (\lambda y. e')[e/x] \triangleq \begin{cases} \lambda y. e' & \text{if } y = x \\ \lambda y. (e'[e/x]) & \text{if } y \neq x \end{cases}$$
$$(e_0 e_1)[e/x] \triangleq (e_0[e/x]) (e_1[e/x])$$

Consider $e \triangleq \lambda x. \lambda y. (x^2 + 2 \cdot y + 5)$ and $e' \triangleq y$.

An Attempt at Substitution



$$y[e/x] \triangleq \begin{cases} e & \text{if } y = x \\ y & \text{if } y \neq x \end{cases} \quad (\lambda y. e')[e/x] \triangleq \begin{cases} \lambda y. e' & \text{if } y = x \\ \lambda y. (e'[e/x]) & \text{if } y \neq x \end{cases}$$
$$(e_0 e_1)[e/x] \triangleq (e_0[e/x]) (e_1[e/x])$$

Consider $e \triangleq \lambda x. \lambda y. (x^2 + 2 \cdot y + 5)$ and $e' \triangleq y$. Their application $(e e')$:

$$\begin{aligned} e e' &= (\lambda x. \lambda y. (x^2 + 2 \cdot y + 5)) y \\ &= (\lambda x. \lambda y. (x^2 + 2 \cdot y + 5))[y/x] \\ &= \lambda y. ((x^2 + 2 \cdot y + 5)[y/x]) \\ &= \lambda y. (y^2 + 2 \cdot y + 5) \end{aligned}$$

An Attempt at Substitution



$$y[e/x] \triangleq \begin{cases} e & \text{if } y = x \\ y & \text{if } y \neq x \end{cases} \quad (\lambda y. e')[e/x] \triangleq \begin{cases} \lambda y. e' & \text{if } y = x \\ \lambda y. (e'[e/x]) & \text{if } y \neq x \end{cases}$$
$$(e_0 e_1)[e/x] \triangleq (e_0[e/x]) (e_1[e/x])$$

Consider $e \triangleq \lambda x. \lambda y. (x^2 + 2 \cdot y + 5)$ and $e' \triangleq y$. Their application $(e e')$:

$$\begin{aligned} e e' &= (\lambda x. \lambda y. (x^2 + 2 \cdot y + 5)) y \\ &= (\lambda x. \lambda y. (x^2 + 2 \cdot y + 5))[y/x] \\ &= \lambda y. ((x^2 + 2 \cdot y + 5)[y/x]) \\ &= \lambda y. (y^2 + 2 \cdot y + 5) \end{aligned}$$

Note: the previously free y is now **captured** by the binder λy !

Capture-Avoiding Substitution



A revised definition of substitution:

$$y[e/x] \triangleq \begin{cases} e & \text{if } y = x \\ y & \text{if } y \neq x \end{cases}$$

$$(\lambda y. e')[e/x] \triangleq \lambda z. ((e'[z/y])[e/x])$$

if $\underbrace{z \notin \text{fv}(\lambda y. e') \cup \text{fv}(e) \cup \{x\}}_{z \text{ is a fresh identifier: not used anywhere}}$

$$(e_0 \ e_1)[e/x] \triangleq (e_0[e/x] \ e_1[e/x])$$

Capture-Avoiding Substitution



Example

Consider again $e \triangleq \lambda x. \lambda y. (x^2 + 2 \cdot y + 5)$ and $e' \triangleq y$.

Their application $(e \ e')$ with capture-avoiding substitution:

$$\begin{aligned} e \ e' &= (\lambda x. \lambda y. (x^2 + 2 \cdot y + 5)) \ y \\ &= (\lambda y. (x^2 + 2 \cdot y + 5)) [y/x] \\ &= \lambda z. ((x^2 + 2 \cdot y + 5) [z/y] [y/x]) \\ &= \lambda z. ((x^2 + 2 \cdot z + 5) [y/x]) \\ &= \lambda z. (y^2 + 2 \cdot z + 5) \end{aligned}$$



The End